# Camunda 8 - Getting Started with Microservice Orchestration

## Business Requirement

We will be using a fictitious organization; Camundanzia, as the scenario for this course.

### Camundanzia

**Scenario**

The Camundanzia insurance company would like to introduce process automation throughout their organization. The first process they would like to automate is their **Payment** process.

Camundanzia has developed an eCommerce website (Webshop) which allows insurance partners to order Camundanzia branded products such as pens, notepads and water bottles for marketing events. Camundanzia partners must purchase these items using a credit card.

**Partner** purchases made using a credit card are processed automatically without the need for approval by an **Accountant**.

### User Story

Two simple User Stories to capture Camundanzia's requirements can be found below:

**User Story**

- As an **Accountant**
- I want all **Partner** purchases to be made using a credit card
- So that all purchases can be made without human intervention

**User Story**

- As a **Partner**
- I want to purchase Camundanzia goods using a credit card
- So that I have an efficient purchasing experience

### Acceptance Criteria

We will take an MVP approach to developing an application that meets these requirements.

The initial acceptance criteria that we will be working to are:

1. **Partner** orders are processed using the **Payment Process**
2. The **Payment Process** requires a *Payment Request* which includes the *Reference*, *Amount* and *Credit Card Details*
3. The **Payment Process** processes orders automatically using a **Credit Card Service** i.e. without human intervention

## Iterative Development

### Iterative Development

Camunda is well suited to an iterative development approach; applications can be easily revised and upgraded without affecting in-flight processes.

# Outline Solution

We now have a clear set of requirements and acceptance criteria for the Camundanzia webshop *Payment Process*.

The sections below will provide you with an overview of how to develop a **Microservice Orchestration** solution that meets these requirements.

## Camundanzia

### Scenario

Imagine that you have recently joined Camundanzia as a new Developer and have been assigned to the Camunda project team. Your first assignment is to implement the *Payment Process* using the requirements from the previous lesson.

Fortunately for you, a more experienced Camunda Developer has drafted an outline solution for you. Your task is to implement a solution based upon the outline that has been provided.

## Outline Solution

The more experienced Developer has provided an overview of the work that must be completed for each of the stated acceptance criteria Both the configuration that you, the Developer, must carry out and the Camunda components that provide the necessary capabilities are covered by this.

*Process Order using Payment Process*

The core of the solution is provided by being able to execute a **Process Instance** in Camunda. In order to execute a **Process Instance** we need to create a **Process Definition** which models the business process.

- **Process Definitions** are created using the Modeler
- **Process Instances** are executed using the Zeebe Engine
- **Process Instances** can be monitored using Operate

### Create Process Definition

Create a **Process Definition** that represents the *Payment Process*.

### Create Charge Credit Card Task

Create a **Task** which will be used to request payment from the *Credit Card Service*.

*Call the Credit Card Service*

The *Credit Card Service* should be called from the *Payment Process* once it reaches the *Charge Credit Card* task. This can be achieved in Camunda by creating a **Job Worker** which calls the *Credit Card Service*.

**Create a Job Worker**

Create a **Job Worker** which calls the *Credit Card Service* to make a payment.

*Create Payment Request*

The *Credit Card Service* requires a *Payment Request* in order to automatically process the transaction.

**Create a Client**

Create a **Client** that starts the *Payment Process* with a *Payment Request* (Reference, Amount & Credit Card Details)

**Handle Process Data**

Handle the **Process Data** that has been provided to the **Process Instance** so that the *Credit Card Service* processes the payment correctly and without human intervention

## Development Plan

Now that we have identified the components and features of Camunda which will be used to implement our solution, we can create a plan of the work that needs to be performed and the tools that will be required.

- **Create a Process Definition**
  - This will be configured using the Desktop Modeler
- **Deploy the Process Definition**
  - This will be performed from the Desktop Modeler
- **Create a Job Worker**
  - This will be performed from your preferred IDE e.g. Eclipse, IntelliJ
- **Test the Process**
  - This will be performed from your preferred IDE, Desktop Modeler and Operate

## Create Client Credentials

We now need to create a set of **Client Credentials** so that we can access our **Cluster**.

**Client Credentials**

In order to interact with the Zeebe Engine in the cloud, every client application must authenticate itself. Authentication against Zeebe is managed using an OAuth flow.

### Create Client Credentials

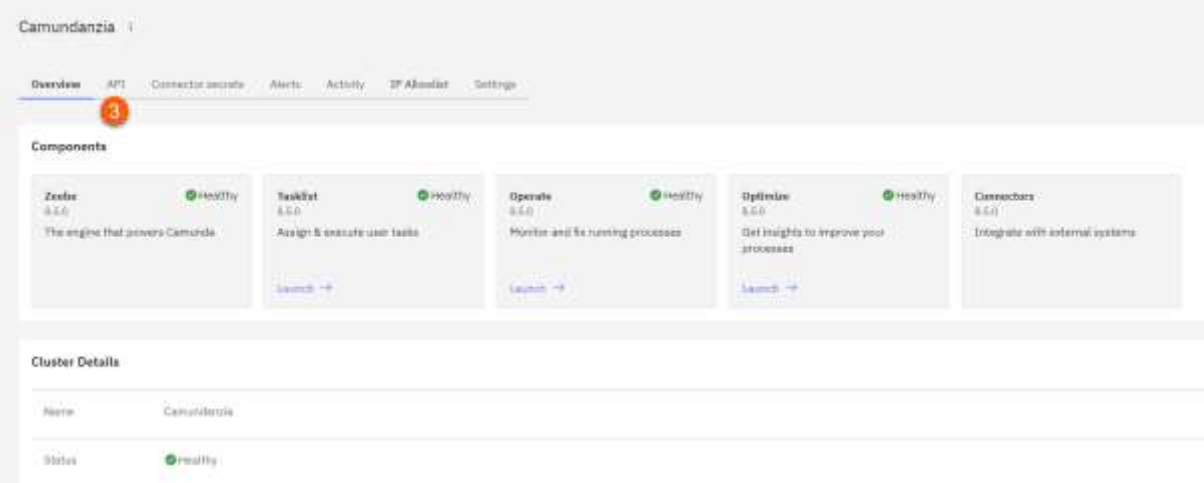
**Existing Client Credentials**

This section can be skipped if you already have created a set of Client Credentials for your Cluster in Camunda SaaS

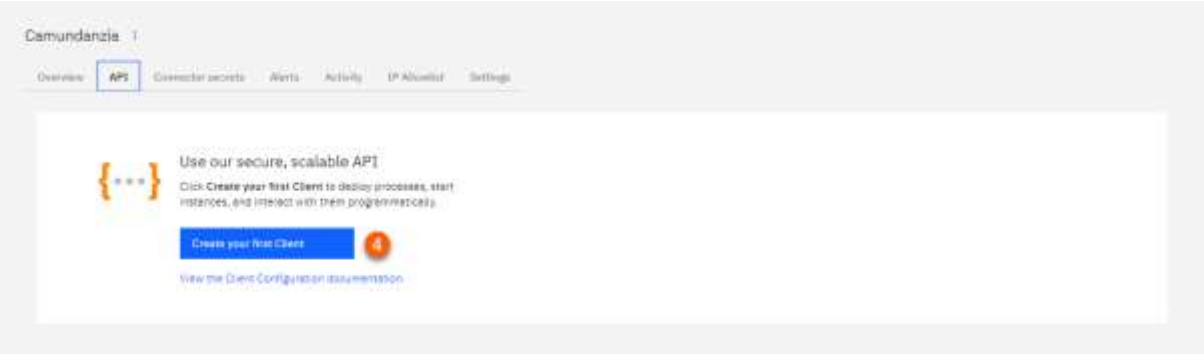
Follow the steps below to create a set of **Client Credentials** for your **Cluster**:

1. Navigate to **Clusters** from the top menu of the Camunda Console.
2. Click on the Camundanzia cluster from the list that appears

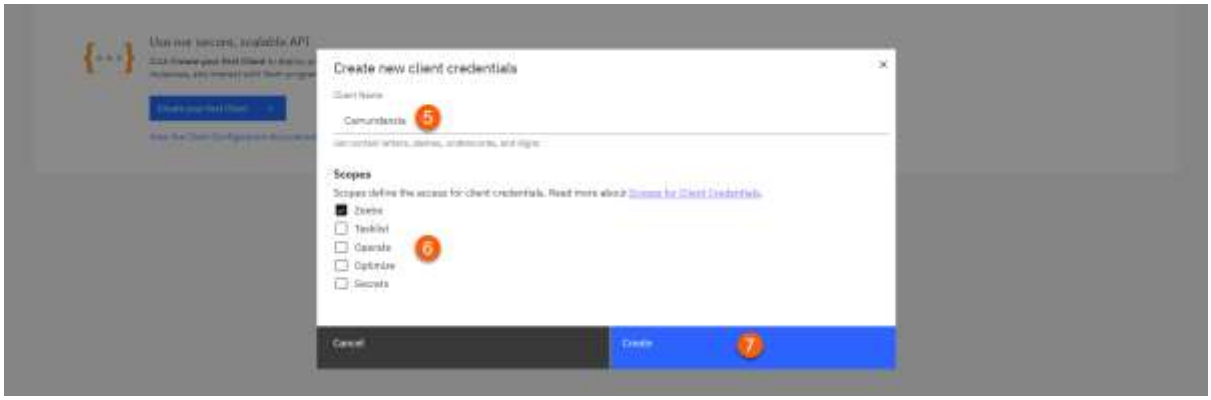
3. Click on the **API** tab from the **Cluster Details** page




4. Click on the **Create your first Client** button to launch the **Create New Client Credentials** dialog box




5. Type a **Name** for the **Client Credentials**. For this course we will be using Camundanzia.
6. Select an appropriate **Scope** for the **Client Credentials**. For this course we will be using Zeebe




## What is a Scope?

The **Scope** for **Client Credentials** limits the level of access that is available with those credentials. For Camunda, the available scopes control access to the following:

1. **Zeebe** - Access to the Zeebe gRPC API
2. **Tasklist** - Access to the Tasklist GraphQL API
3. **Operate** - Access to the Operate REST API
4. **Optimize** - Access to the Optimize REST API
5. **Secrets** - Access cluster secrets in a hybrid setup

## Using connectors?

**Connector Runtime** expects valid credentials for Zeebe and Operate according to the connector type:

1. **Outbound** connectors require only one Scope: **Zeebe**.
2. **Inbound** connectors require two Scopes: **Zeebe** and **Operate**

## Principle of Least Privilege

When creating **Client Credentials** you should try to follow the **Principle of Least Privilege** i.e. clients should only have access to the minimum scope required to perform their function.

7. Click **Create** to create the **Client Credentials**
8. A dialog box will be displayed containing both the **Spring Boot** and **Environment Variable** version of your **Client Credentials**

### Spring Boot

Clicking the **Spring Boot** tab will allow you to download the credentials as an `application.properties` file that can be easily used with the Spring Boot Zeebe Client.

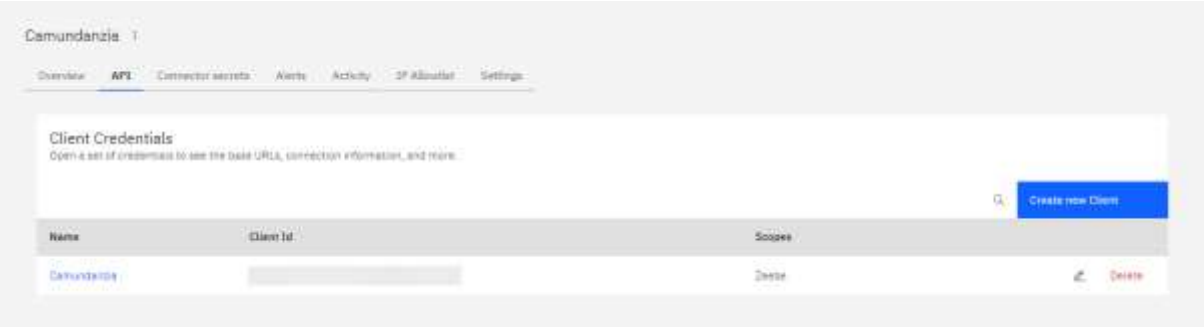
9. Click the **Environment Variables** tab
10. Click the **Download** button
11. A file named `CamundaCloudMgmtAPI-Client-<<Client Credentials Name>>.txt` will be downloaded to your local `Downloads` folder. This file contains the following properties which are required to connect to your **Cluster**.

```
export ZEEBE_ADDRESS='**********.zeebe.camunda.io:443'export
ZEEBE_CLIENT_ID='**********'export
ZEEBE_CLIENT_SECRET='**********'export
ZEEBE_AUTHORIZATION_SERVER_URL='https://login.cloud.camunda.io/oauth/to
ken'export ZEEBE_TOKEN_AUDIENCE='zeebe.camunda.io'export
CAMUNDA_CLUSTER_ID='**********'export CAMUNDA_CLUSTER_REGION='bru-
2'export CAMUNDA_CREDENTIALS_SCOPES='Zeebe'export
CAMUNDA_OAUTH_URL='https://login.cloud.camunda.io/oauth/token'
```

### Save Credentials

Remember to save your credentials before closing the dialog box. You will need to generate new credentials if you forgot to save them at this point.

12. Your new **Client Credentials** will be displayed under the **API** tab once you close the dialog box




### Created Client Credentials

Congratulations! You have now created a set of **Client Credentials** which can be used to access our **Cluster**.

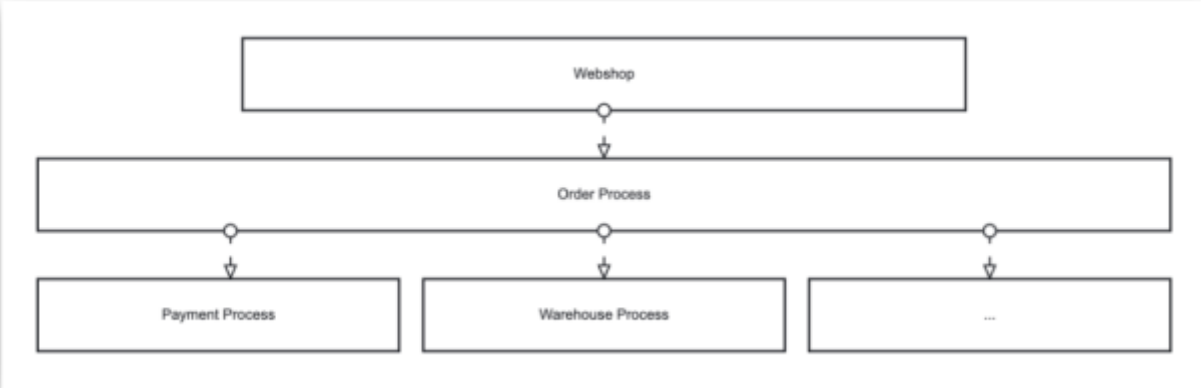
# Create Process

We will now create a new **Process Definition** (BPMN Diagram) which will be used to model the *Payment Process*.

### Payment Process

The *Payment Process* is one part of the *Order Process* which will be started whenever a **Partner** places an order from the webshop. The *Order Process* orchestrates all of the tasks which need to be completed to fulfil the order.

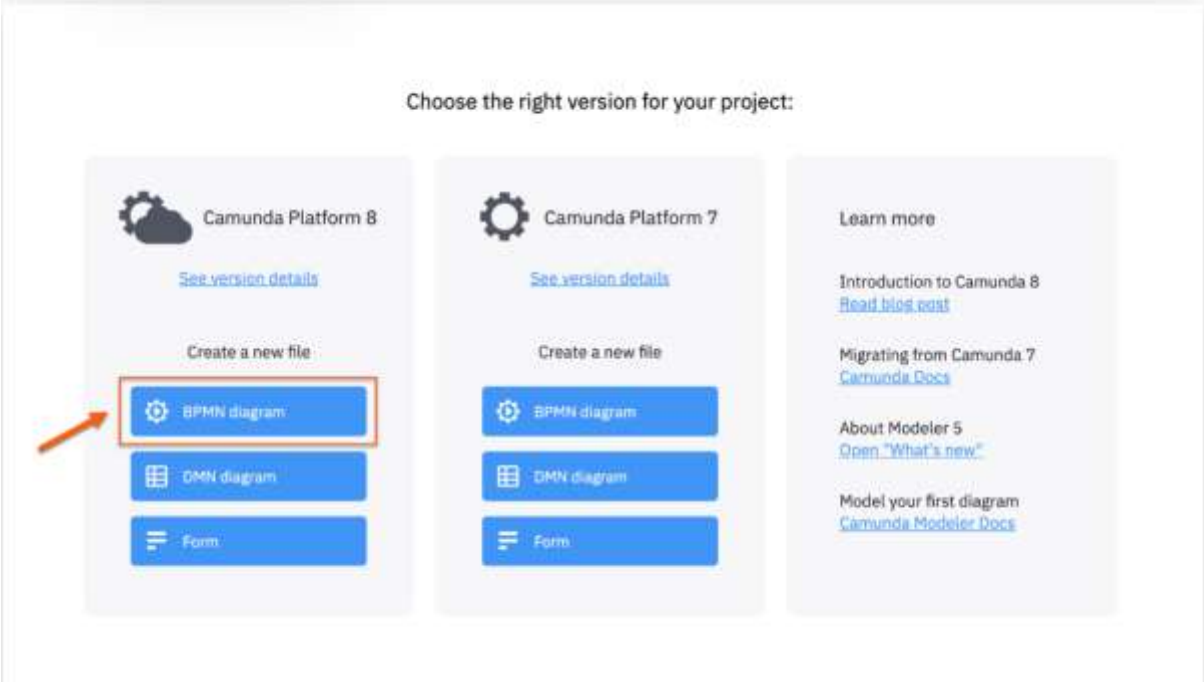
The *Order Process* contains an activity to invoke the *Payment Process* which is the focus of this course.




## Create Process Definition

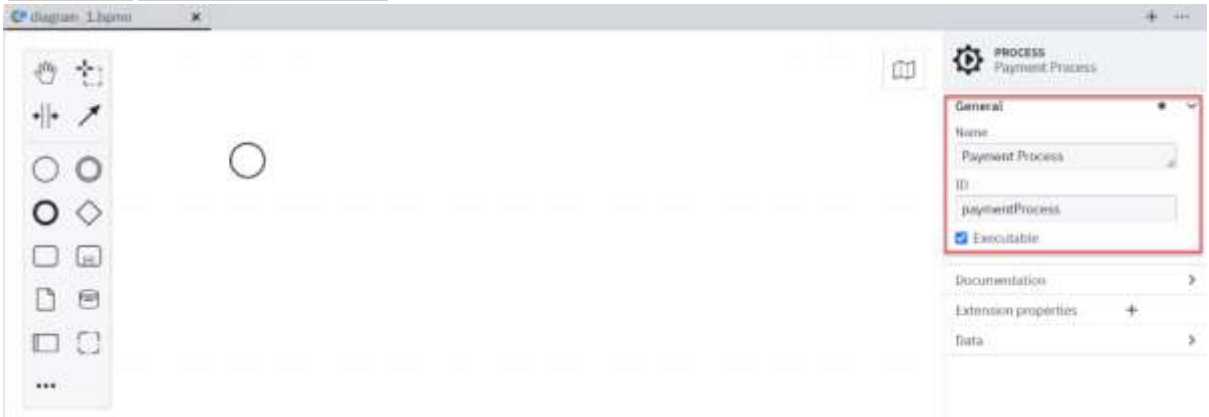
Create a new **Process Definition** by following the steps below:

1. Launch the **Desktop Modeler**
2. Create a new **Camunda 8 BPMN Diagram** by clicking the button on the **Modeler** launch page




3. This will open a new **BPMN Diagram** in the **Modeler**. In the properties panel on the right-hand side of the screen, set the **Name** and **ID** as follows:

| Name | Value |
|---|---|
| **Name** | Payment process |
| **ID** | paymentProcess |




## Naming Conventions for BPMN IDs

Define developer-friendly and business-relevant IDs for the process itself, as well as all activities, messages, and errors. Also consider events, gateways, and the sequence flows that carry conditional expressions. Even though IDs are just identifiers, keep in mind that they will show up regularly on the technical level. Meaningful IDs will help a lot.

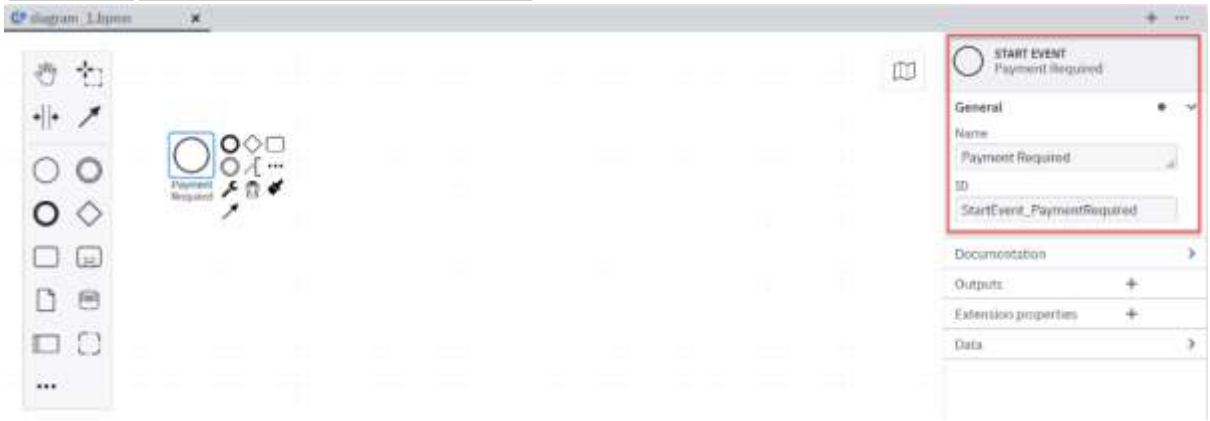
# Model Business Process

*Add Start and End Events*

All **Process Definitions** require at least one **Start Event** and at least one **End Event**

1. Click on the **Start Event** in the process and then expand the **General** tab in the properties panel on the right-hand side of the screen. Set the **Name** and **ID** as follows:

| Name | Value |
|---|---|
| **Name** | Payment Required |
| **ID** | StartEvent_PaymentRequired |



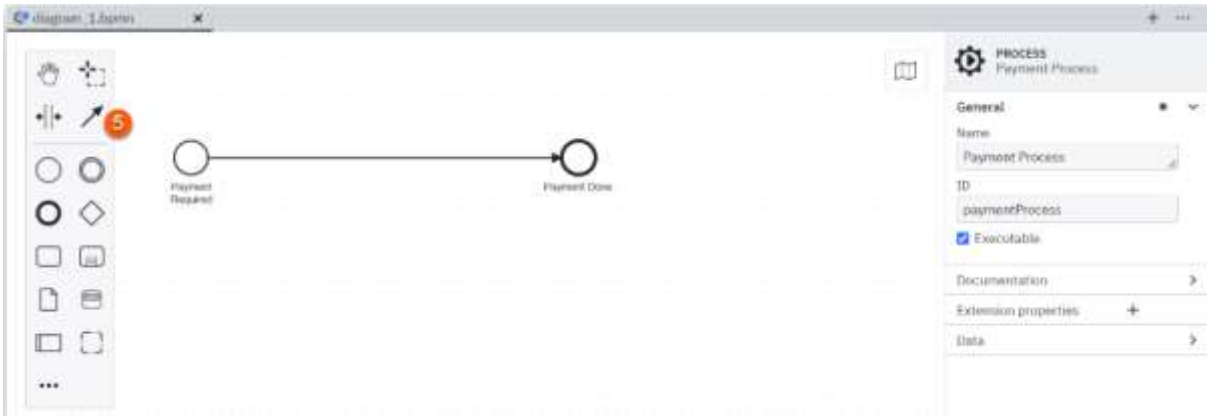

2. Click on the **End Event** from the **BPMN Palette** to add a new element to the diagram.
3. Click on the right-hand side of the diagram to place the **End Event**
4. Set the **Name** and **ID** for the **End Event** as follows:

| Name | Value |
|---|---|
| **Name** | Payment Done |
| **ID** | EndEvent_PaymentDone |

**Naming Events**

Wherever possible, name an event using an object and a verb reflecting a state. Always try to describe which state an object is in when the process is about to leave the event.

5. Connect both events by activating the **Global Connect Tool** in the pallet and dragging a **Sequence Flow** from the **Start Event** to the **End Event**

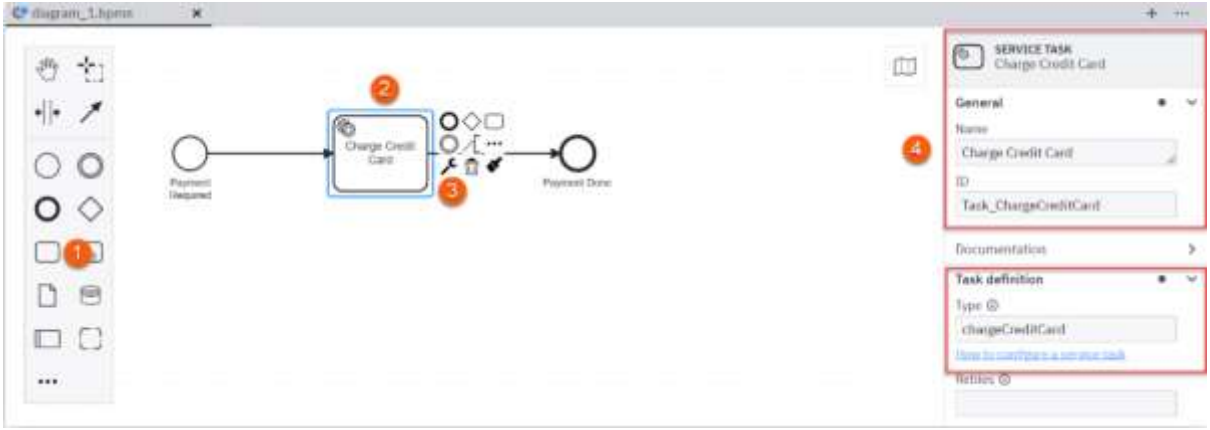



*Add a Task*

For the first draft of our process, we will create a single **Service Task**

1. Click on the **Task** from the **BPMN Palette** to add a new element to the diagram
2. Click the middle of the previously created **Sequence Flow** to place the **Task** on the diagram
3. Click on the **Change Type (Wrench)** icon in the toolbox and then select **Service Task** from the context menu that appears
4. Set the **Name**, **ID** and **Type** for the **Service Task** as follows:

| **Name** | **Value** |
|---|---|
| **Name** | Charge Credit Card |
| **ID** | Task_ChargeCreditCard |
| **Type** | chargeCreditCard |




## What is a Task Definition

A **Service Task** must have a **Task Definition**; the Task Definition defines which **Job Workers** can execute the **Task**.

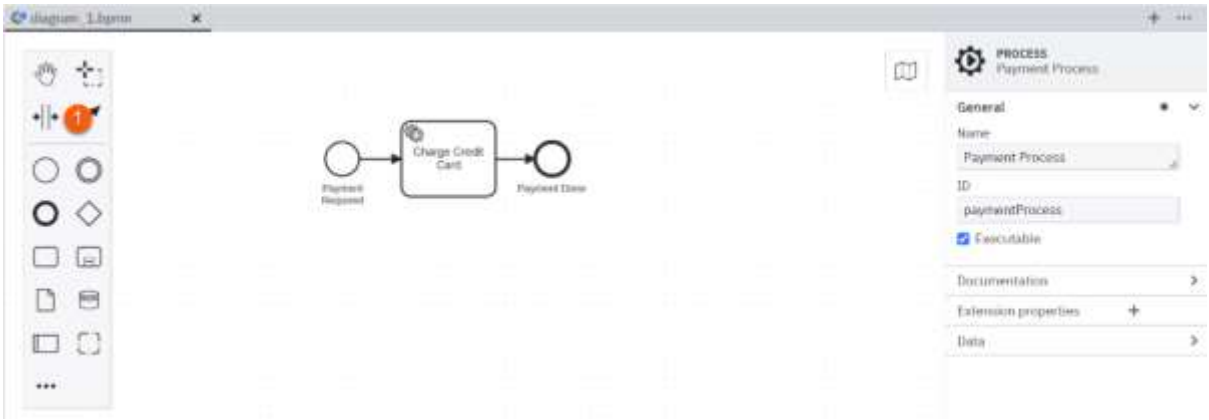
When a **Service Task** is encountered in a **Process Instance**, a corresponding **Job** is created. The **Process Instance** stops here and waits until the **Job** has been completed.

A **Job Worker** can subscribe to a **Job Type**, process the **Job** and complete them using one of the **Zeebe Clients**. Once the **Job** is complete, the **Service Task** is complete and the **Process Instance** continues.

### *Arrange Diagram*

Finally, we can arrange the diagram using the **Create / Remove Space Tool**.

1. Click on the **Create / Remove Space Tool** in the **BPMN Palette** on the left-hand side of the screen
2. Use the tool to evenly space the BPMN elements on the diagram. Your diagram should look similar to the one below when you have finished




## Readable Diagrams

We create visual process models to better understand, discuss, and remember processes. Hence, it is crucial that models are easy to read and understand.

3. Click **File -> Save File** from the top menu to save your process diagram in a folder of your choice
4. Name it as **paymentProcess.bpmn**.

**Created a Process Definition**

Congratulations! You have now created a **Process Definition** which captures the *Payment Process*.

# Deploy Process

We will now need to deploy our **Process Definition** to the previously created **Cluster** so that we can start a **Process Instance**.

**What is a Deployment?**

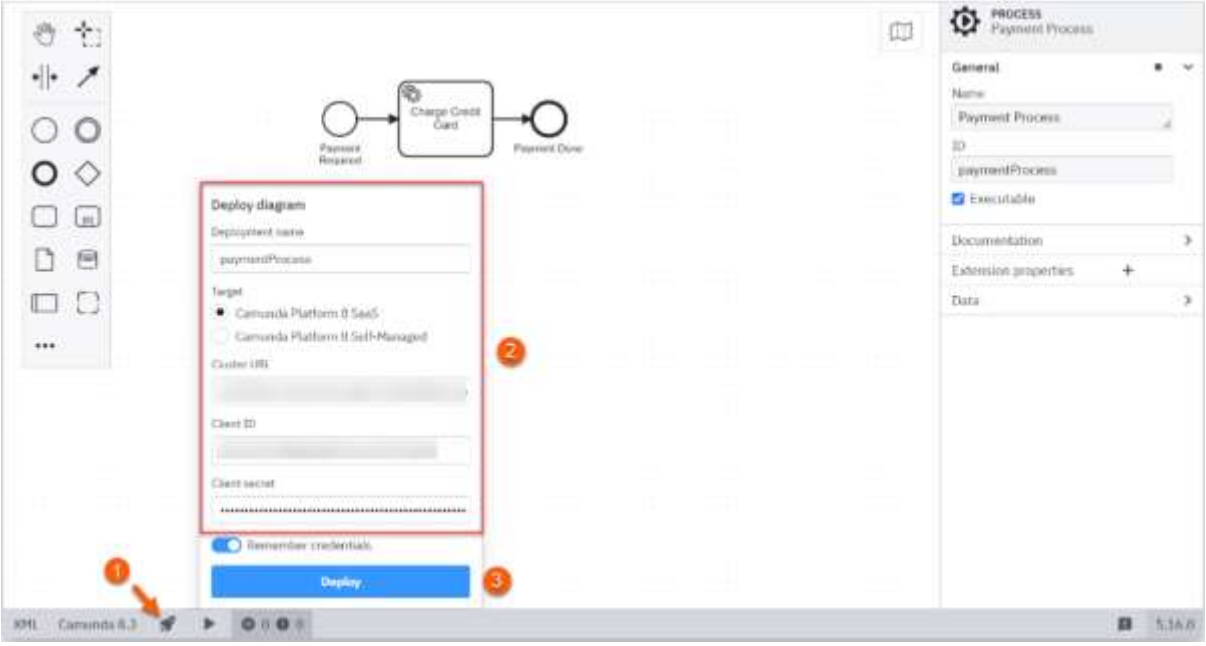
A **Deployment** allows the **Process Definition** to be executed by the **Zeebe Engine**.

## Deploy Process Definition

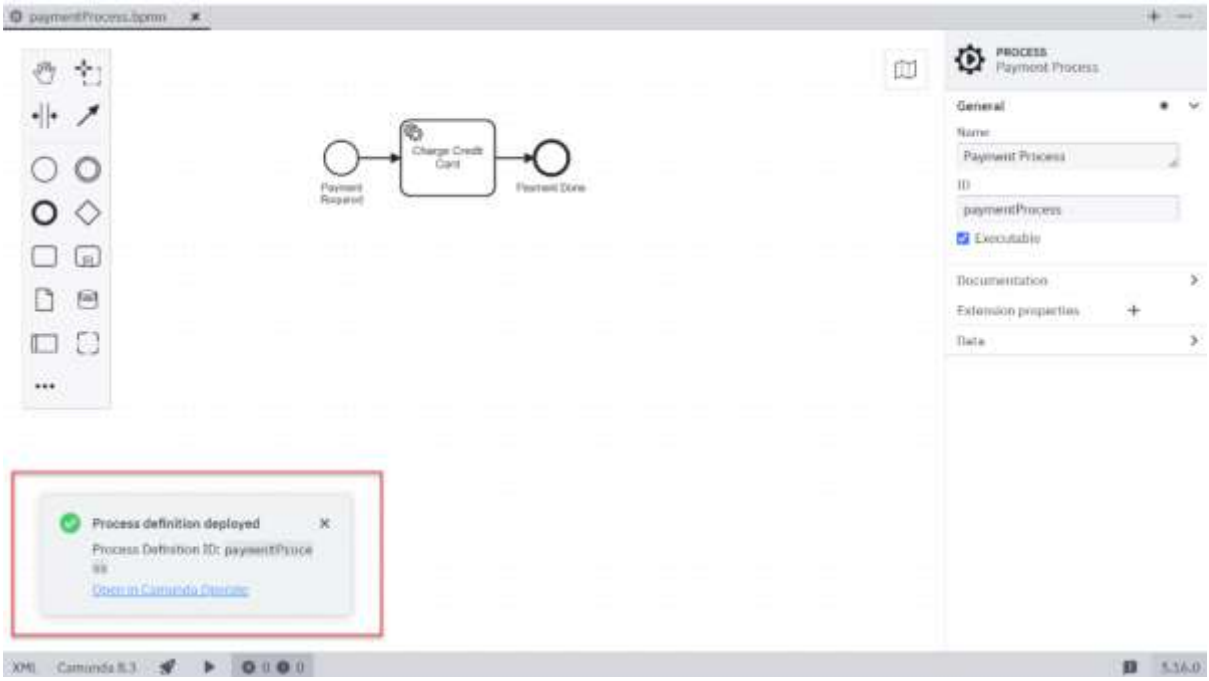
1. Click the **Deploy Current Diagram** button from the **Status Bar** of the Camunda Modeler
2. This will open a **Deploy Diagram** dialog box which should be populated using the values below. The `ZEEBE_` environment variables can be obtained from the `CamundaCloudMgmtAPI-Client-Camundanzia.txt` which you downloaded in an earlier lesson

| Name | Value |
|---|---|
| **Deployment Name** | paymentProcess |
| **Target** | Camunda 8 SaaS |
| **Cluster URL** | `ZEEBE_ADDRESS` |
| **Client ID** | `ZEEBE_CLIENT_ID` |
| **Client Secret** | `ZEEBE_CLIENT_SECRET` |

3. Click the **Deploy** button to deploy your process model to your **Cluster**




4. You should see a **Deployment Succeeded** message if everything has been configured correctly




**Deployment Error**

Check that you have populated the correct values from your **Client Credentials** if you receive an error message instead of a success message.

## View the Deployment in Operate

Next, we can take a look into Operate to confirm that the deployment has been successful.

**What is a Deployment?**

**Operate** is a tool for monitoring and troubleshooting process instances running in the **Zeebe Engine**.

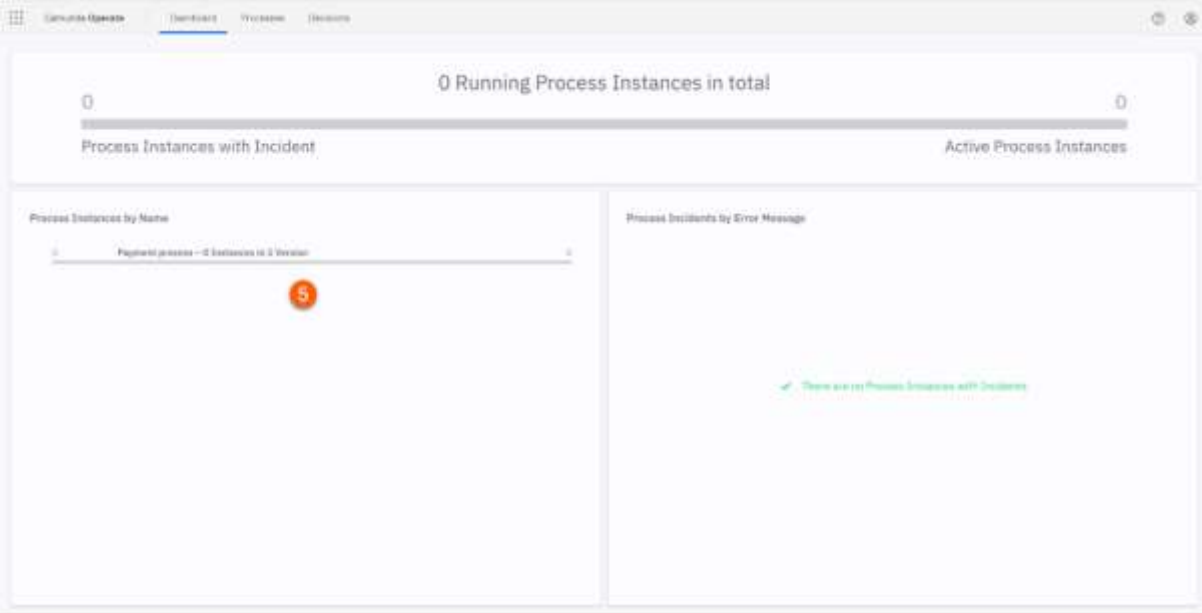
In addition to providing visibility into active and completed process instances, Operate also makes it possible to carry out key operations such as resolving incidents, and updating process instance variables.

### *Launch Operate*

Launch **Operate** by following the steps below:

1. From the **Console**, select **Clusters**
2. Select *Camundanzia* from the **Clusters** page
3. Click the **Launch** link next to **Operate** in the **Cluster Details** page

4. After a few seconds, the **Operate** Dashboard will be displayed
5. You will also be able to see that the *Payment Process* is visible in the Dashboard



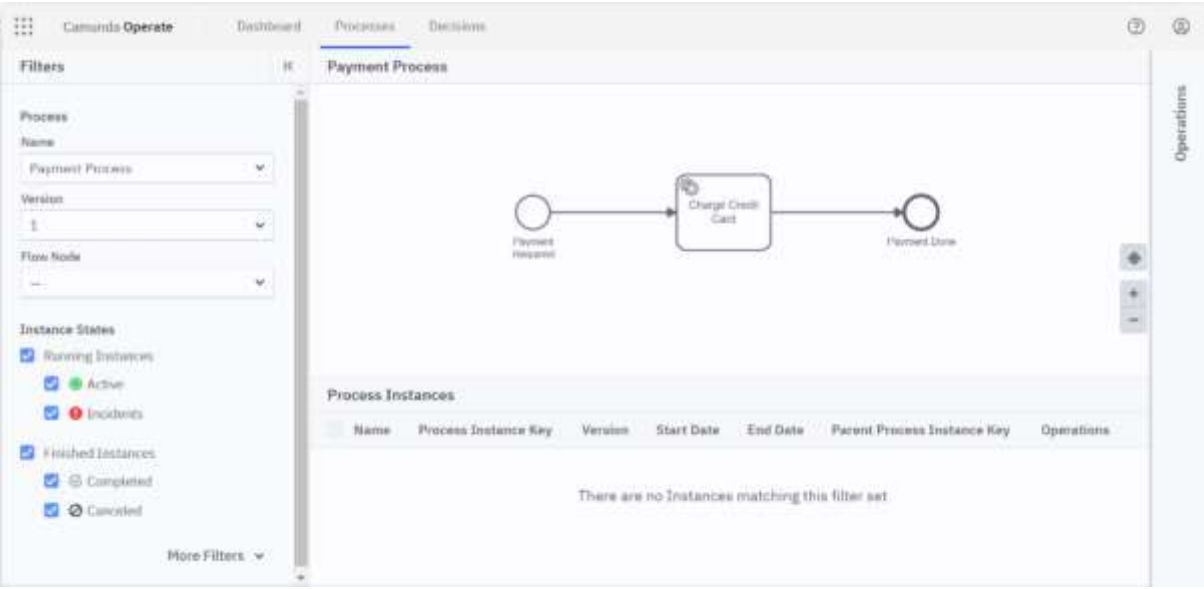

### *View Process Instances*

We can now view the process instances in **Operate** by following the steps below:

**What is a Process Instance?**

One execution of a **Process Definition** is called a **Process Instance**. Many **Process Instances** can be launched from one **Process Definition**.

1. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
2. **Operate** will not show any active process instances at this point. However, we can see that version 1 of our *Payment* process has been successfully deployed




**Deployed a Process Definition**

Congratulations! You have now deployed the *Payment Process* **Process Definition** into the **Zeebe Engine**.

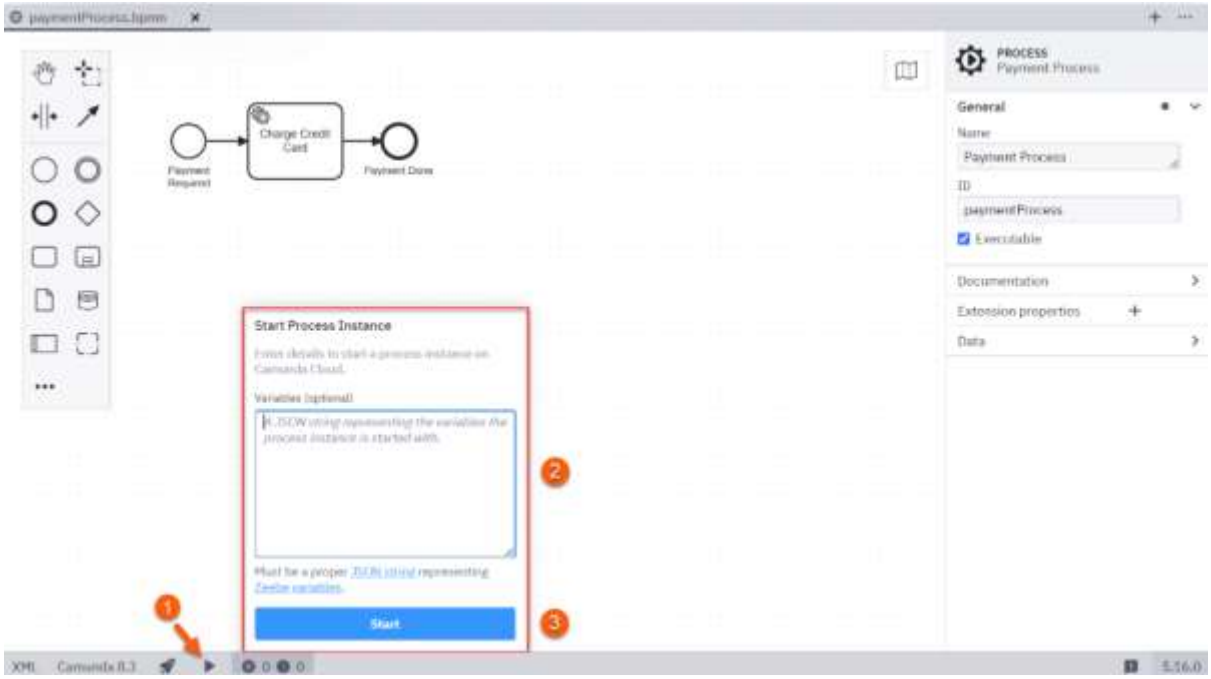
# Start Process

We will now need to execute our **Process Definition** for the first time; this is also known as starting a **Process Instance**.

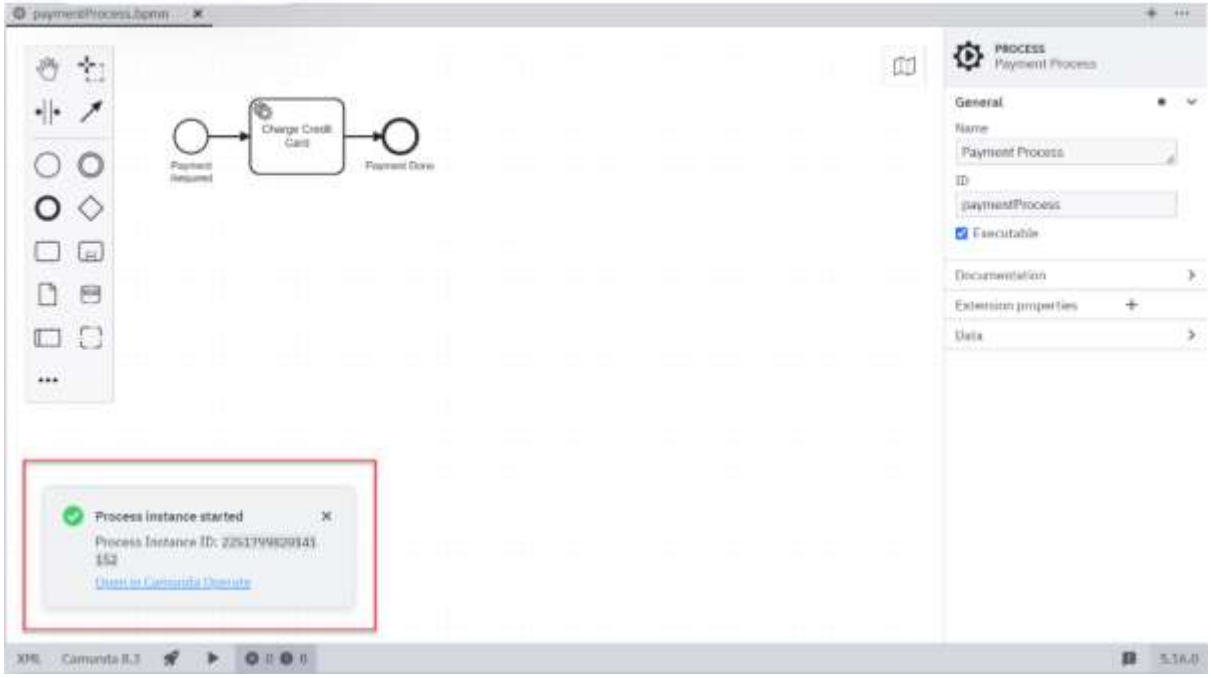
## Start a Process Instance

Start an instance of the *Payment Process* by following the steps below:

1. Select the **Start Current Diagram** button in the **Status Bar** of the Modeler
2. This will open a **Start Process Instance** dialog box
3. Click on the **Start** button without making any changes to the **Variables** section



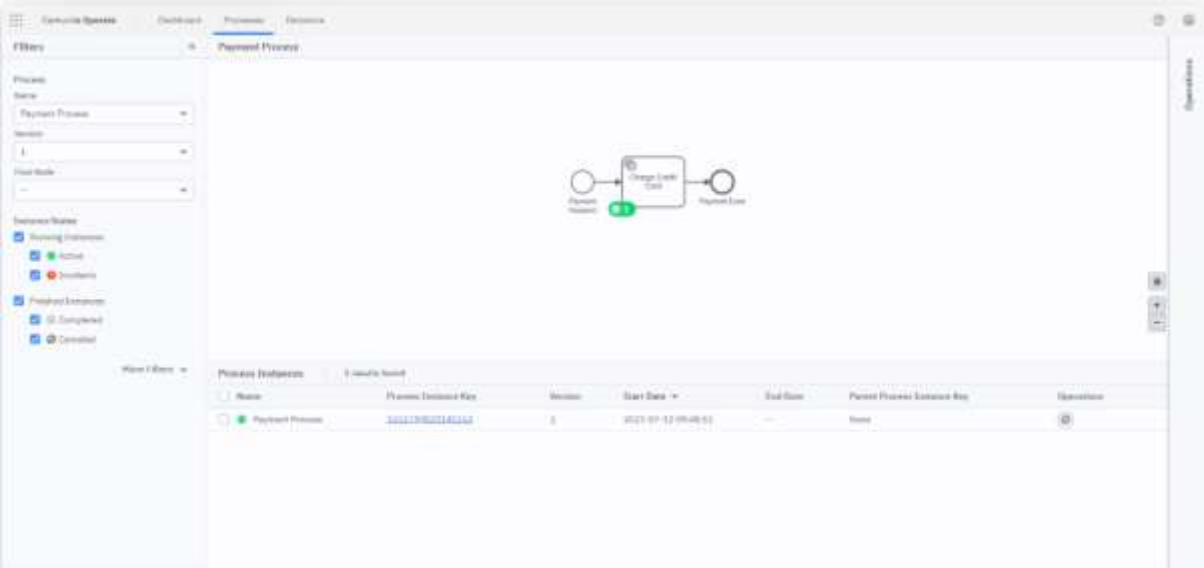

4. You should get a **Process instance started** message




## View Process Instances

We can now view the process instances in **Operate** by following the steps below:

1. Launch **Operate** by following the steps in the previous lesson. Alternatively, refresh the window if you still have **Operate** open
2. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
3. **Operate** will now show a running instance of our *Payment Process*. The completion of the *Charge Credit Card* **Service Task** is still pending; you can see this from the presence of the green marker against the **Service Task**



**Started a Process Instance**

Congratulations! You have now started an instance of the *Payment Process*.

## Create Job Worker

We will now create a Job Worker which will be used to perform the *Charge Credit Card* task in the process that we have modeled.

**What is a Job Worker?**

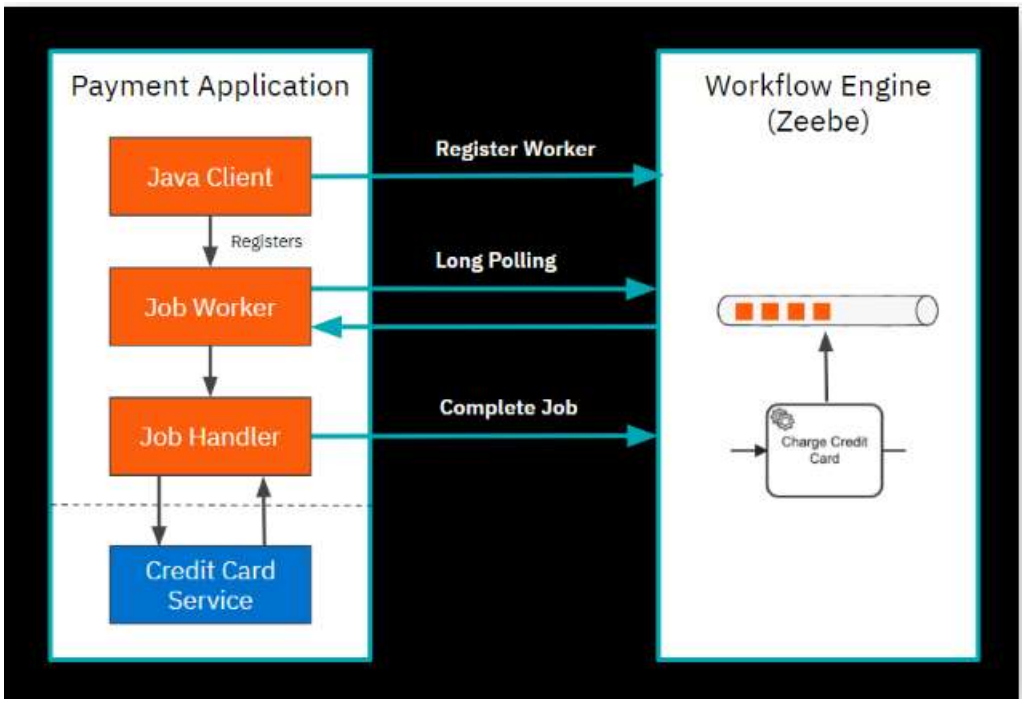
A **Job Worker** is a service capable of performing a particular task in a process. **Job Workers** are used to manage the interactions between the Zeebe Engine and the outside world.

In this course we will be orchestrating a *Credit Card Service* from the *Payment Process*. The *Payment Process* is represented by a **Process Definition** and is executed as a **Process Instance** by Zeebe. The *Credit Card Service* is external to Zeebe and could be hosted anywhere; it is the **Job Worker** that connects the *Credit Card Service* to the *Payment Process*.

# Payment Application

For this course, we will develop a *Payment Application* which will contain the following elements:

- Zeebe Client - The Zeebe Client allows you to interact with Zeebe using the API
- Job Worker - The Job Worker is responsible for requesting Jobs of a defined type from the Zeebe Engine
- Job Handler - The Job Handler is responsible for performing the actions required to complete a Job when it has been acquired by the Job Worker
- Credit Card Service - This is a simple, mock representation of a Credit Card Service




## Zeebe Clients

Zeebe Clients allow applications to perform the following:

- Deploy a Process Definition
- Start and Cancel a Process Instance
- Activate Jobs and Complete Jobs
- Update Process Variables

Camunda provides several official clients for interacting with the Zeebe API. In this course we will be using the Java Client to create the *Payment Application*.

# Create Client (Java)

As we will be using Java to develop our **Job Worker**, we will start by creating a new Maven project.

## Create a Maven Project

Create a new Maven project by following the steps below (Command Line or IDE):

**Create Maven Project using the Command Line**

1. Start a Command Prompt e.g. `Terminal` or `PowerShell`
2. Change to an appropriate directory e.g. your IDE workspace
3. Create a Maven Project using the following command

```
mvn archetype:generate \
-DgroupId=com.camunda.academy \
-DartifactId=c8-getting-started-microservice-orchestration-lab \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DarchetypeVersion=1.4 \
-DinteractiveMode=false
```

4. Maven will generate the new project and will output something similar to the following if everything has been successful

```
[INFO] Project created from Archetype in dir:
C:\DevTools\Workspace\eclipse\camunda-academy\c8-getting-started-
microservice-orchestration-lab
[INFO] ------------------------------------------------------------------
--------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------
--------
[INFO] Total time:  3.457 s
[INFO] Finished at: 2022-12-09T14:44:55Z
[INFO] ------------------------------------------------------------------
--------
```

5. Change to the newly created project using `cd c8-getting-started-microservice-orchestration-lab`
6. Confirm that the project can be built using `mvn clean package`

```
[INFO] ------------------------------------------------------------------
--------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------
--------
[INFO] Total time:  2.926 s
[INFO] Finished at: 2022-12-09T14:54:20Z
[INFO] ------------------------------------------------------------------
--------
```

### Create Maven Project using an IDE

1. Start your preferred IDE e.g. Eclipse or IntelliJ
2. Create a new Maven project with the following properties

| Name | Value |
|---|---|
| **groupId** | com.camunda.academy |
| **artifactId** | c8-getting-started-microservice-orchestration-lab |

| Name | Value |
| --- | --- |
| **archetypeArtifactId** | maven-archetype-quickstart |
| **archetypeVersion** | 1.4 |
| **version** | 1.0-SNAPSHOT |

3. Confirm that the project can be built using `mvn clean package`

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.926 s
[INFO] Finished at: 2022-12-09T14:54:20Z
[INFO] ------------------------------------------------------------------------
```

## Add the Java Client Dependency

A **Job Worker** can be thought of as a specialized Client. **Clients** connect to Camunda 8 via gRPC, a high-performance, open source and universal RPC protocol.

Camunda provides several official clients based upon this API which provide convenience functions (e.g. thread handling for Job Workers) on top of the core API.

We will now add the Java Client dependency into our Maven project to gain access to the convenience functions that have been provided.

1. Add the following dependency to your `pom.xml`

```
<dependency>
  <groupId>io.camunda</groupId>
  <artifactId>zeebe-client-java</artifactId>
  <version>${zeebe.version}</version>
</dependency>
```

**Java Client Version**

You should use the latest stable version of the Java Client from the Maven Repository.

At the time of writing, the latest stable version is `8.6.0`.

**Spring Boot**

If you are building a Spring or Spring Boot application, you may want to use the **Spring Zeebe** dependency instead of handling the lifecycle and configuration of the Java client yourself.

The **Spring Zeebe** client is a community maintained client which is not officially supported by Camunda.

2. Confirm that the project can be built using `mvn clean package`

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.926 s
[INFO] Finished at: 2022-12-09T14:54:20Z
[INFO] ------------------------------------------------------------------------
```

## Connect to the Cluster

We will now connect to the **Cluster** to ensure that everything is working as expected before implementing the handler for our **Job Worker**.

### *Configure Connection Properties*

For the purposes of simplicity, we will configure the connection properties for our **Job Worker** as Java constants. In a real-world application these properties would be picked up from an `application.properties` file or environment variables based upon the development framework that you are using.

1. Create a new `PaymentApplication` Class with a `main` method in the `com.camunda.academy` package
2. Add the following constants to your Class

```java
private static final String ZEEBE_ADDRESS = "[ZEEBE_ADDRESS]";
private static final String ZEEBE_CLIENT_ID = "[ZEEBE_CLIENT_ID]";
private static final String ZEEBE_CLIENT_SECRET =
"[ZEEBE_CLIENT_SECRET]";
private static final String ZEEBE_AUTHORIZATION_SERVER_URL =
"[ZEEBE_AUTHORIZATION_SERVER_URL]";
private static final String ZEEBE_TOKEN_AUDIENCE =
"[ZEEBE_TOKEN_AUDIENCE]";
```

3. Replace each of the constants in square brackets with the values from your environment.

**Connection Properties**

- **ZEEBE_ADDRESS** - The address to reach the Zeebe Cluster, i.e. a standalone or embedded gateway or a proxy / load balancer sitting in front of a Zeebe Cluster
- **ZEEBE_CLIENT_ID** - The unique identifier of the Client for the OAuth2 provider
- **ZEEBE_CLIENT_SECRET** - The pre-shared secret for the OAuth2 provider
- **ZEEBE_AUTHORIZATION_SERVER_URL** - The URI from which an Access Token can be requested from

- **ZEEBE_TOKEN_AUDIENCE** - The audience of the Client token for the OAuth2 provider

*Configure the Credentials Provider*

We now need to create an `OAuthCredentialsProvider` so that our **Client** can authenticate with the **Cluster**.

1. Add the following snippet to the `main` method to instantiate the `OAuthCredentialsProvider`:

```
final OAuthCredentialsProvider credentialsProvider =
    new OAuthCredentialsProviderBuilder()
        .authorizationServerUrl(ZEEBE_AUTHORIZATION_SERVER_URL)
        .audience(ZEEBE_TOKEN_AUDIENCE)
        .clientId(ZEEBE_CLIENT_ID)
        .clientSecret(ZEEBE_CLIENT_SECRET)
        .build();
```

*Create a Zeebe Client*

We can now create a **Zeebe Client** which will connect to our **Cluster** using the address and credentials that have been made available through our constants:

1. Add the following snippet to the `main` method beneath the instantiation of the `OAuthCredentialsProvider`:

```
final ZeebeClient client =
    ZeebeClient.newClientBuilder()
        .gatewayAddress(ZEEBE_ADDRESS)
        .credentialsProvider(credentialsProvider)
        .build();
```

**Close the Client**

The **Zeebe Client** connection should be closed when it is no longer needed. Whilst the snippet above will work, a **try-with-resources** statement would be a better implementation here to ensure that resources are cleaned up.

2. Modify the snippet above so that the **Zeebe Client** will be cleaned up after use

```
try (final ZeebeClient client =
        ZeebeClient.newClientBuilder()
            .gatewayAddress(ZEEBE_ADDRESS)
            .credentialsProvider(credentialsProvider)
            .build()) {
} catch (Exception e) {
    e.printStackTrace();
}
```

*Make a Request*

Before we implement the functionality for our **Job Worker**, we can make a simple request to retrieve the Topology of the **Cluster**.

1. Modify the **Zeebe Client** snippet so that it now reflects the following:

```
try (final ZeebeClient client =
        ZeebeClient.newClientBuilder()
            .gatewayAddress(ZEEBE_ADDRESS)
            .credentialsProvider(credentialsProvider)
            .build()) {
    System.out.println("Connected to: " +
client.newTopologyRequest().send().join());
} catch (Exception e) {
    e.printStackTrace();
}
```

**Remember to Send**

Requests sent to the **Zeebe API** from a **Client** are asynchronous; **Clients** use the same pattern to make requests against the API for consistency and ease of use.

1. Obtain a new **Command** object from the **Client**
2. Configure the **Command** object (specific to each Command e.g. CreateInstanceCommand)
3. **Send** the **Command** to the Zeebe API
4. **Join** the **Future** object to wait for and handle the response

Always remember to Send the **Command** after it has been configured and Join the **Future** to wait for and handle the response.

*Test the Connection*

We can now test the connection between our **Client** and the **Cluster**:

1. If you now run the `PaymentApplication` from your IDE or Maven, you should get a connection to the **Cluster** and a response similar to the below:

```
Connected to Camunda 8:
TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1, host='zeebe-1.zeebe-
broker-service.5f400cdf-a978...
```

**Connected to a Cluster**

Congratulations! You have now developed a simple **Client** that is able to connect to a Camunda **Cluster** using the **Zeebe API**.

# Create Job Handler (Java)

We will now convert our generic **Client** into a **Job Worker** by adding a **Job Handler**.

## Create the Credit Card Service

We will start by creating a simple *Credit Card Service* which we will invoke from the *Payment Process*. For the purposes of simplicity, this will be a Mock Service that will log a message to show that a request has been received.

1. Create a new `CreditCardService` class in the `com.camunda.academy.service` package
2. Add a new `chargeCreditCard` method to `CreditCardService`
3. Add a simple logging statement to `chargeCreditCard`

```
public class CreditCardService {

    public void chargeCreditCard() {
        System.out.println("Charging Credit Card");
    }
}
```

## Create the Credit Card Service Handler

We will now need to create a **Handler** that will invoke the *CreditCardService* each time a **Process Instance** reaches a **Service Task** with the type of *chargeCreditCard*.

This `CreditCardServiceHandler` will implement the `JobHandler` interface and we will supply an implementation for the `handle` method which will invoke the `chargeCreditCard` method in the `CreditCardService`.

1. Create a new `CreditCardServiceHandler` class which implements `JobHandler` in the `com.camunda.academy.handler` package
2. Implement the `JobHandler` interface by adding a new `handle` method to the `CreditCardServiceHandler`
3. Instantiate and invoke the `CreditCardService` from the **Handler**

```
public class CreditCardServiceHandler implements JobHandler {

    CreditCardService creditCardService = new CreditCardService();

    @Override
    public void handle(JobClient client, ActivatedJob job) throws
Exception {
        creditCardService.chargeCreditCard();
    }
}
```

4. After the `CreditCardService` has been invoked, we need to tell Zeebe that the **Job** has been completed. Add the following statement to the end of the `handle` method:

```
client.newCompleteCommand(job.getKey()).send().join();
```

**Complete Job**

The **Complete Job** command informs **Zeebe**, that the specified **Job** has been completed and that the **Process Instance** can continue.

See Complete Job for further information.

## Create the Worker

We now need to create a **Job Worker** using the **Job Handler** that was implemented in the previous section. The **Job Worker** will process any task with the *chargeCreditCard* type and will execute a **Job Handler** for that task.

1. Replace the `System.out.println("Connected to: " + client.newTopologyRequest().send().join());` statement from `PaymentApplication` with the following:

```
final JobWorker creditCardWorker =
    client.newWorker()
        .jobType("chargeCreditCard")
        .handler(new CreditCardServiceHandler())
        .timeout(Duration.ofSeconds(10).toMillis())
        .open();
```

2. The code snippet above registers a new **Job Worker** for **Jobs** of a given type. In this instance, the worker is registered against the *chargeCreditCard* type (`jobType` parameter).
3. After registration, the Zeebe Broker activates any available **Jobs** and assigns them to this **Worker**.
4. The worker will then process and complete each job using its **Handler**. In this instance, we are using the `CreditCardServiceHandler` to process and complete the **Job** (`handler` parameter).
5. In the event of a timeout (due to an unforeseen issue), the **Worker** will release the **Job** which will allow it to be picked up and processed by another **Worker**. In this instance, we have set a timeout of 10s (`timeout` parameter).
6. Finally, to keep things simple in this course, we will wait 10 seconds before exiting the application. This will allow our **Worker** to acquire and complete the pending **Job**. Add a call to `Thread.sleep(10000);` immediately after the worker has been created.

```
final JobWorker creditCardWorker =
    client.newWorker()
        .jobType("chargeCreditCard")
        .handler(new CreditCardServiceHandler())
        .timeout(Duration.ofSeconds(10).toMillis())
        .open();
Thread.sleep(10000);
```

**Multiple Workers**

It is possible to run multiple workers for a single task type in Camunda. This provides scalability when handling very high volumes of process instances and resilience if a worker should fail.

For this reason, it is important that tasks are exclusively reserved for processing by a worker; this prevents the job from being executed by multiple workers concurrently. This reservation is acquired when the worker activates a job; it is released either when the job has been completed or when the worker times out.

## Start the Worker

Finally, we can now start our **Job Worker** which will allow it to process the *Charge Credit Card* task in our waiting process instance.

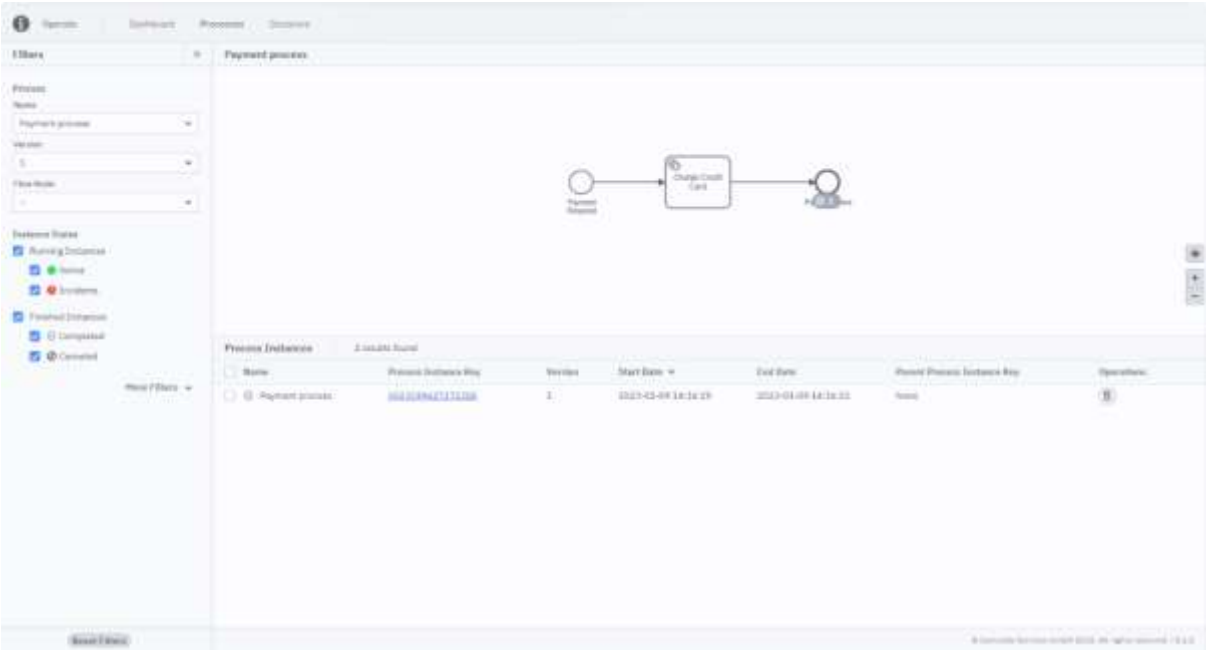
1. If you now run the *PaymentApplication* from your IDE or Maven, you should see output similar to the below for each process instance that you have started.

```
Charging Credit Card
```

## View Completed Process Instances

We can now view the completed process instances in **Operate** by following the steps below:

1. Launch **Operate** by following the steps in the previous lesson. Alternatively, refresh the window if you still have **Operate** open.
2. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
3. **Operate** will now show a completed instance of our *Payment Process*. The previously pending *Charge Credit Card* **Service Task** will now be shown as completed. You can see this from the presence of the grey marker against the **End Event**.



**Job Handled**

Congratulations! You have now created a **Job Worker** that has completed the *Charge Credit Card* task by invoking an external service.

# Start Process with Variables (Modeler)

To continue building out our example, we will need to provide a *Payment Request* to the *Payment Process* so that we can pass it to the *Credit Card Service*.

### Process Variables

**Process Variables** are part of a **Process Instance**. Variables can be passed in at the start of a process and can be accessed and modified during the execution of a process. Variables can also be accessed and modified by a **Job Worker**.

You can find more information here.

## Start Process with Variables (Modeler)

You can use the Desktop Modeler to start a **Process Instance** with a *Payment Request* by following the steps below.

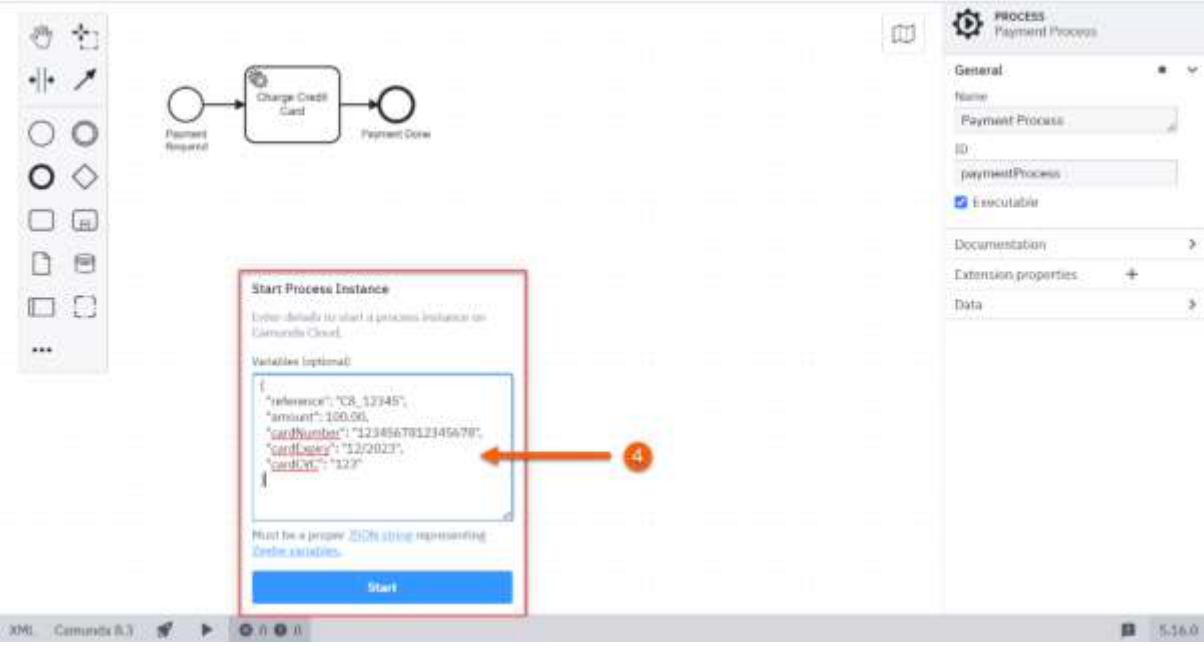
### Payment Request

For the purposes of this course we will define a *Payment Request* as follows:

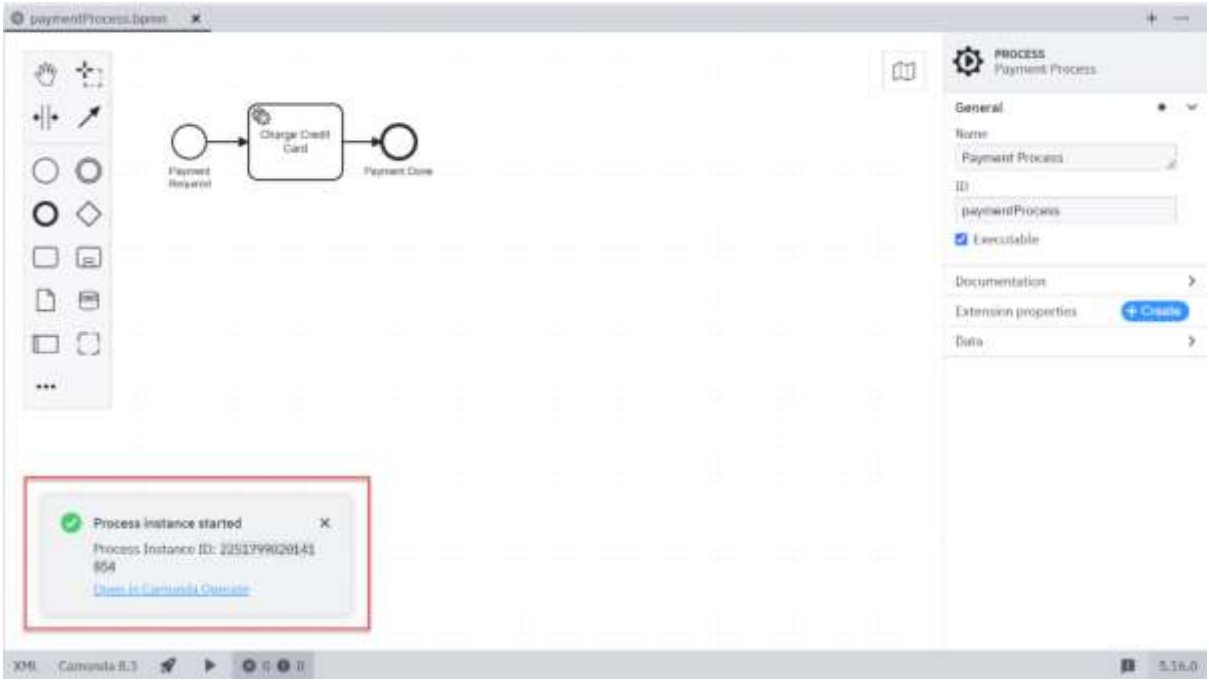
- reference (String) - A unique Payment Reference
- amount (Double) - The Payment Amount
- cardNumber (String) - The 16-digit Card Number
- cardExpiry (String) - The Card Expiry Date MM/YYYY
- cardCVC (String) - The Card Security Number

1. Ensure that the *PaymentApplication* is not running.
2. Select the **Start Current Diagram** button in the **Status Bar** of the Modeler
3. This will open a **Start Process Instance** dialog box
4. Enter the following JSON into the **Variables** field and then click on the **Start** button

```
{
  "reference": "C8_12345",
  "amount": 100.00,
  "cardNumber": "1234567812345678",
  "cardExpiry": "12/2023",
  "cardCVC": "123"
}
```



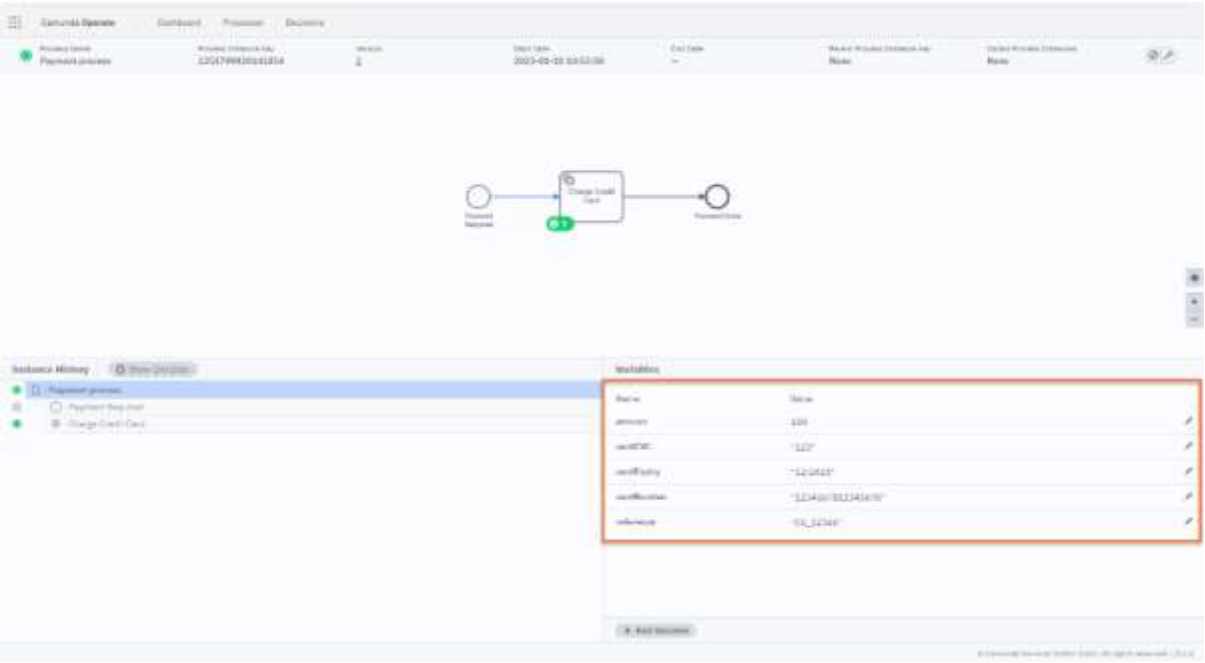

5. You should get a **Process instance started** message.




## View Process Instances

We can now view the process instance in **Operate** by following the steps below:

1. Launch **Operate** by following the steps in the previous lesson. Alternatively, refresh the window if you still have **Operate** open.
2. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
3. **Operate** will now show a running instance of our *Payment Process* along with the variables representing our *Payment Request*. The completion of the *Charge Credit Card* **Service Task** is still pending; you can see this from the presence of the green marker against the **Service Task**.



**Started a Process Instance**

Congratulations! You have now started an instance of the *Payment Process* with a *Payment Request* using the **Desktop Modeler**.

# Start Process with Variables (Client)

Starting process instances using the **Desktop Modeler** is useful for testing purposes but in the real world it is much more useful to start a process instance programmatically.

## Start Process with Variables (Client)

We can use the existing *PaymentApplication* to start a **Process Instance** with a *Payment Request* by following the steps below.

**Multiple Applications**

For the purposes of simplicity in this course, we will add all client functionality into a single Java *PaymentApplication*. However, in a real world implementation, it is much more likely that this functionality would be distributed across multiple applications and potentially implemented using different languages / frameworks.

1. Ensure that the *PaymentApplication* is not running
2. Create a `Map` of the **Process Variables** that will be used to start the **Process Instance** by adding the following snippet to the `main` method above the registration of `client.newWorker` i.e. the first statement of the `try` block:

```
final Map<String, Object> variables = new HashMap<String, Object>();
variables.put("reference", "C8_12345");
variables.put("amount", Double.valueOf(100.00));
variables.put("cardNumber", "1234567812345678");
```

```
variables.put("cardExpiry", "12/2023");
variables.put("cardCVC", "123");
```

3. Add a new **Create Instance Command** beneath the map of process variables

```
client.newCreateInstanceCommand()
    .bpmnProcessId("paymentProcess")
    .latestVersion()
    .variables(variables)
    .send()
    .join();
```

**Create Instance Command**

The **Create Instance Command** command creates and starts an instance of the specified process. The process definition used to create the instance can be specified either using its unique key (as returned by DeployProcess), or using the BPMN process ID and a version. You can read more here.

## Run the Application

We can run our updated *PaymentApplication*. This will create a new **Process Instance** and will then launch a **Job Worker** which will complete the *Charge Credit Card* task and subsequently the process.

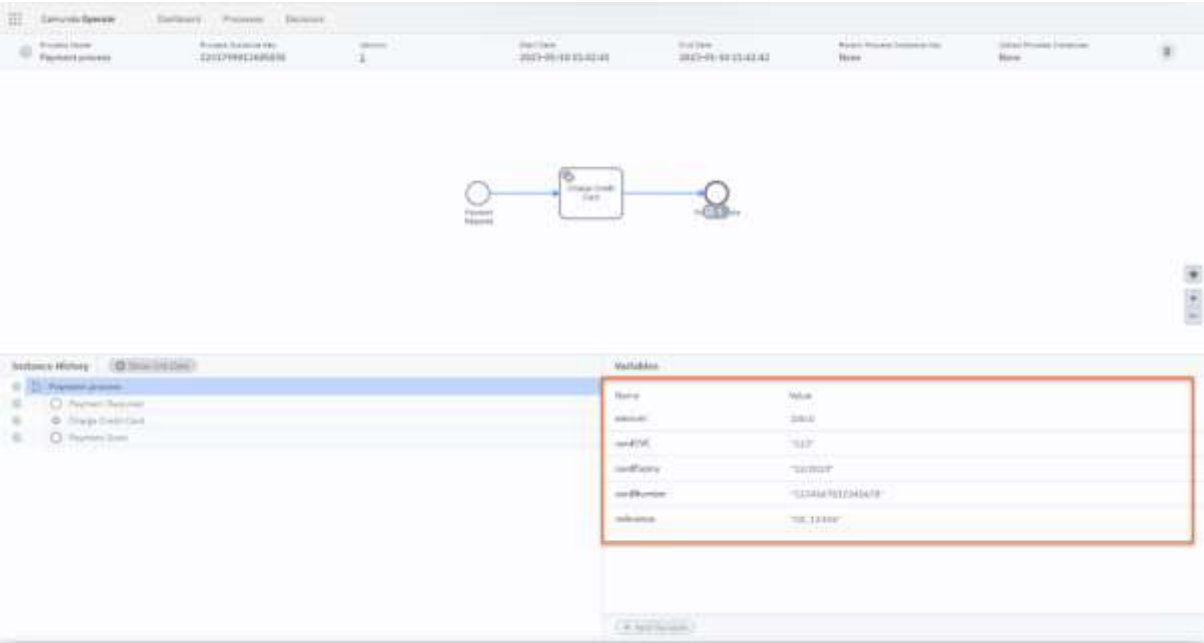
If you now run the *PaymentApplication* from your IDE or Maven, you should see output similar to the below:

```
Charging Credit Card
```

## View Process Instances

We can now view the process instance in **Operate** by following the steps below:

1. Launch **Operate** by following the steps in the previous lesson. Alternatively, refresh the window if you still have **Operate** open
2. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
3. **Operate** will now show a completed instance of our *Payment Process* along with the variables representing our *Payment Request*. The *Charge Credit Card* **Service Task** is now complete; you can see this from the presence of the grey marker against the **End Event**



**Started a Process Instance**

Congratulations! You have now started an instance of the *Payment Process* with a *Payment Request* using the **Java Client**.

# Handle Data in the Job Worker

We can now pass a *Payment Request* into the *Payment Process* using either the **Desktop Modeler** or a **Client**. Next, we need to update our **Job Worker** to retrieve the process variables and modify them accordingly after the job has been completed.

## Update the Credit Card Service

We will start by updating the *Credit Card Service* which is invoked from the *Payment Process*. The `CreditCardService` will now accept the *PaymentRequest* that was passed in at the start of the process and return a confirmation number.

1. Open the `CreditCardService` in the `com.camunda.academy.service` package
2. Modify the `chargeCreditCard` method to `CreditCardService` to accept the parameters representing the *PaymentRequest*
3. Log the *PaymentRequest* that is passed and return a confirmation number

```
public class CreditCardService {

    public String chargeCreditCard(final String reference,
                                   final Double amount,
                                   final String cardNumber,
                                   final String cardExpiryDate,
                                   final String cardCVC) {

        System.out.println("Starting Transaction: " + reference);
        System.out.println("Card Number: " + cardNumber);
        System.out.println("Card Expiry Date: " + cardExpiryDate);
        System.out.println("Card CVC: " + cardCVC);
```

```
            System.out.println("Amount: " + amount);

            final String confirmation =
String.valueOf(System.currentTimeMillis());
            System.out.println("Successful Transaction: " + confirmation);
            return confirmation;
        }
}
```

## Update the Credit Card Service Handler

We will now need to update the **Handler** that will invoke the `CreditCardService` each time a **Process Instance** reaches a **Service Task** with the type of *chargeCreditCard*. This `CreditCardServiceHandler` will now pass the **Process Variables** into the `CreditCardService` and will return the confirmation number from the service into the **Process Instance**.

1. Open the `CreditCardServiceHandler` in the `com.camunda.academy.handler` package
2. Obtain a list of **Process Variables** from the **Job** with the following snippet

```
final Map<String, Object> inputVariables = job.getVariablesAsMap();
final String reference = (String) inputVariables.get("reference");
final Double amount = (Double) inputVariables.get("amount");
final String cardNumber = (String) inputVariables.get("cardNumber");
final String cardExpiry = (String) inputVariables.get("cardExpiry");
final String cardCVC =  (String) inputVariables.get("cardCVC");
```

3. Modify the `chargeCreditCard` method invocation to accept the parameters representing the *PaymentRequest* and return the confirmation number

```
final String confirmation =
      creditCardService.chargeCreditCard(reference, amount, cardNumber,
cardExpiry, cardCVC);
```

4. Create a `Map` of the **Process Variables** that will be returned to the **Process Instance**

```
final Map<String, Object> outputVariables = new HashMap<String,
Object>();
outputVariables.put("confirmation", confirmation);
```

5. Modify the **Complete Command** to also return the confirmation number from the Credit Card Service

```
client.newCompleteCommand(job.getKey()).variables(outputVariables).send
().join();
```

6. The **Handler** should now look similar to the following:

```
public class CreditCardServiceHandler implements JobHandler {

      CreditCardService creditCardService = new CreditCardService();

      @Override
      public void handle(JobClient client, ActivatedJob job) throws
Exception {

            final Map<String, Object> inputVariables =
job.getVariablesAsMap();
            final String reference = (String)
inputVariables.get("reference");
            final Double amount = (Double)
inputVariables.get("amount");
            final String cardNumber = (String)
inputVariables.get("cardNumber");
            final String cardExpiry = (String)
inputVariables.get("cardExpiry");
            final String cardCVC =  (String)
inputVariables.get("cardCVC");

            final String confirmation =
creditCardService.chargeCreditCard(reference, amount, cardNumber,
cardExpiry, cardCVC);

            final Map<String, Object> outputVariables = new
HashMap<String, Object>();
            outputVariables.put("confirmation", confirmation);


client.newCompleteCommand(job.getKey()).variables(outputVariables).send
().join();
      }
}
```

## Run the Application

If you now run the *PaymentApplication* from your IDE or Maven, you should see output similar to the below:

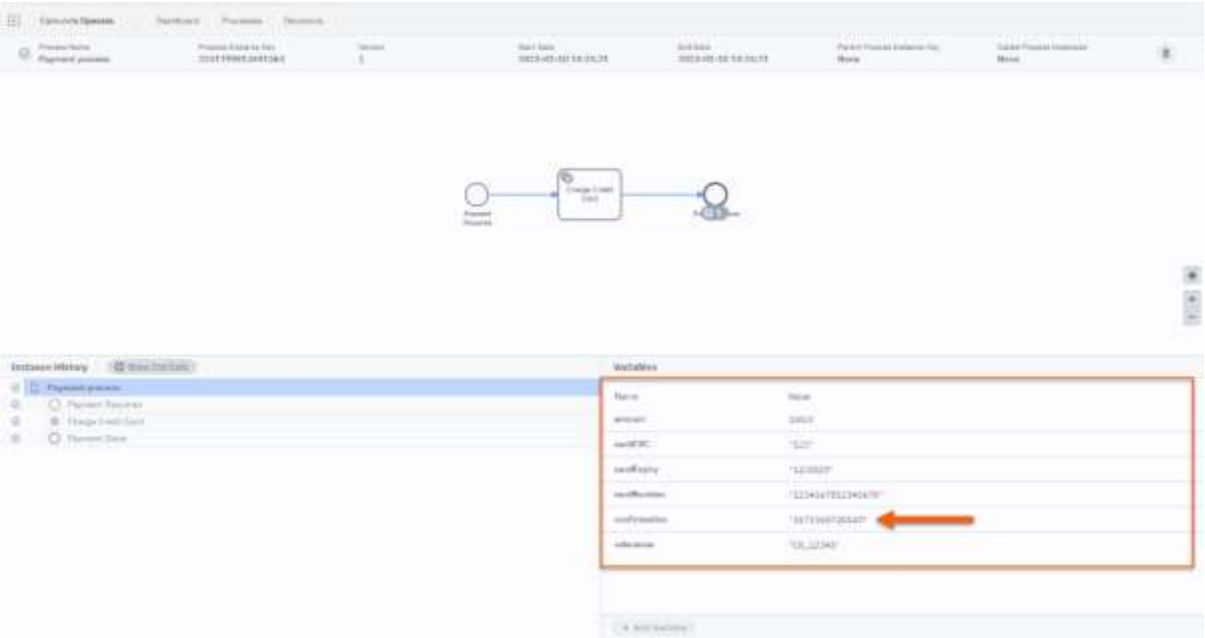
```
Starting Transaction: C8_12345
Card Number: 1234567812345678
Card Expiry Date: 12/2023
```

```
Card CVC: 123
Amount: 100.0
Successful Transaction: 1673360720543
```

## View Process Instances

Finally, view the process instance in **Operate** by following the steps below:

1. Launch **Operate** by following the steps in the previous lesson. Alternatively, refresh the window if you still have **Operate** open.
2. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
3. **Operate** will now show a completed instance of our *Payment Process* along with the variables representing our *Payment Request*. The *Charge Credit Card* **Service Task** is now complete; you can see this from the presence of the grey marker against the **End Event**.



**Started a Process Instance**

Congratulations! You have now started an instance of the *Payment Process* that passes **Process Variables** to and from a **Job Worker**.

# Cleanup Environment

## Overview

We have now met all of the business requirements that have been defined in our **Scenario**. We are now in a position where we can perform an end-to-end test of our process to ensure that everything is working as expected.

However, as we have been testing our process definition as we have been developing it, it is likely that we have several incomplete process instances still running. Having incomplete (and

possibly incorrect) process instances running in our environment may make it more difficult to confirm that everything is working as expected in an end-to-end test.

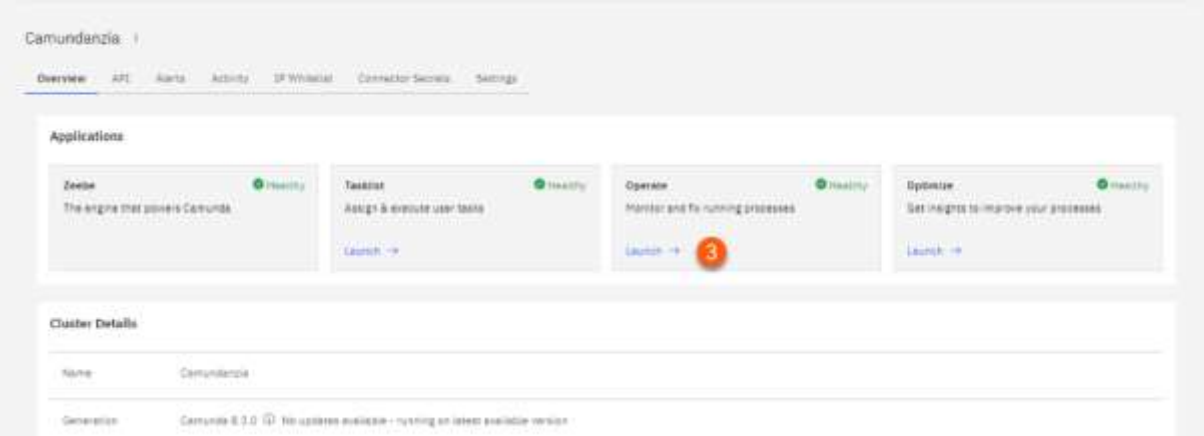
We will now **Cancel** and **Delete** any remaining instances so that we have a completely clean environment in which we can conduct our acceptance tests.

## Cancel & Delete Process Instances

### *Launch Operate*

Launch **Operate** by following the steps below:

1. From the **Console**, select **Clusters**
2. Select *Camundanzia* from the **Clusters** page
3. Click the **Launch** link next to **Operate** in the **Cluster Details** page




4. After a few seconds, the **Operate** Dashboard will be displayed

### *Cancel Process Instances*

We can now **Cancel** the process instances in **Operate** by following the steps below:

1. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
2. On the bottom left of the screen is a series of filters that will allow you to select *Running* or *Finished* process instances
3. Ensure that the *Running* filters are selected and the *Finished* filters are not selected
4. Then, for each **Process Instance** that is shown in the **Instances** panel, click the **Cancel Instance** button to the right-hand side of the process

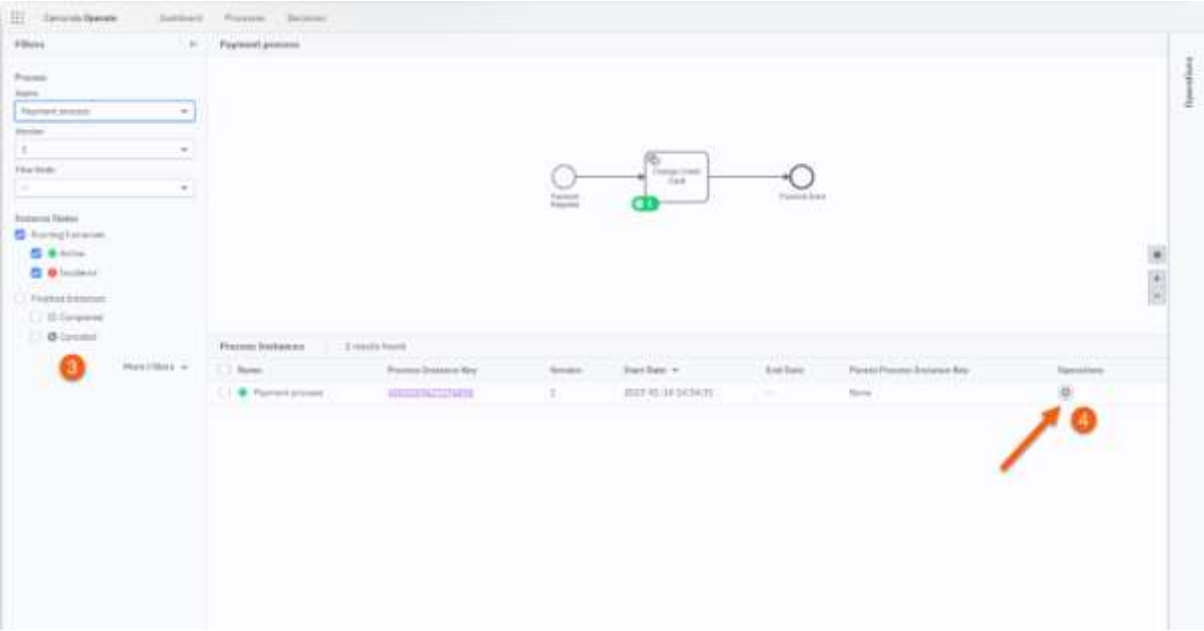
   **No Running Processes**

   This section can be skipped if there are no running processes shown in the **Instances** panel.

5. The **Process Instance** will then be cancelled and will disappear from the **Instances** panel

   **What happened to the Process Instance?**

When you **Cancel** a process in Operate it changes to a stopped state. As the process is no longer running it is not picked up by *Running* filter.



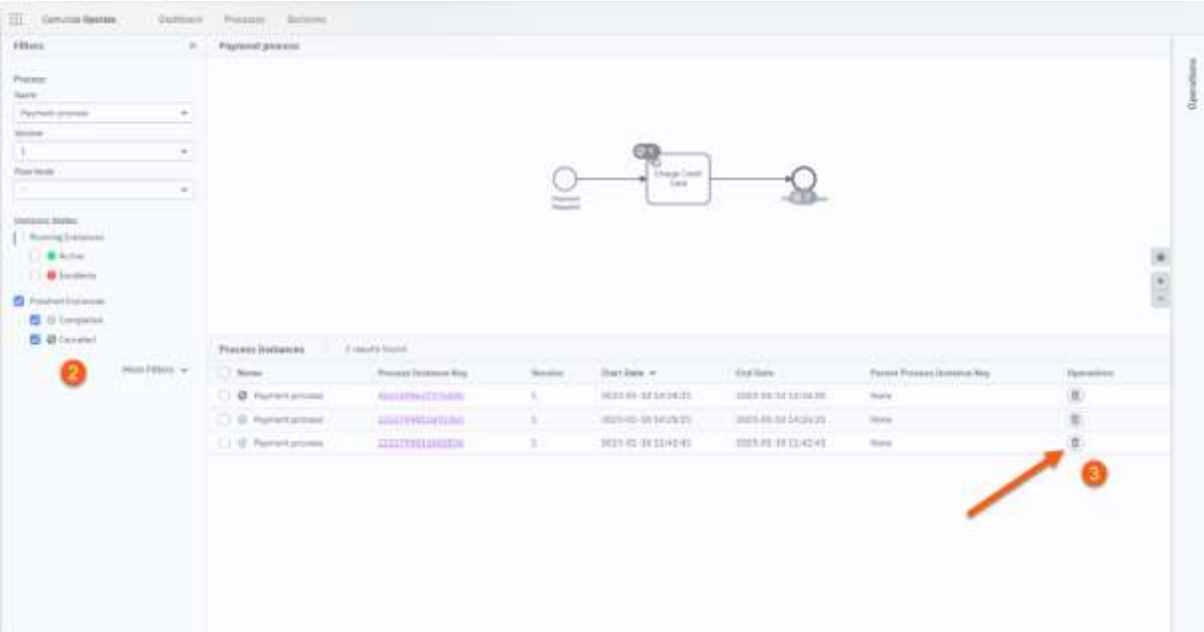
*Delete Process Instances*

We can now **Delete** the finished process instances in **Operate** by following the steps below:

1. On the bottom left of the screen is a series of filters that will allow you to select *Running* or *Finished* process instances
2. Ensure that the *Finished* filters are selected and the *Running* filters are not selected
3. Then, for each **Process Instance** that is shown in the **Instances** panel, click the **Delete Instance** button to the right-hand side of the process
4. The **Process Instance** will then be deleted and will disappear from the **Instances** panel

**What happened to the Process Instance?**

When you **Delete** a process in Operate it is completely removed from the **Zeebe Engine**.



**Cleaned the Environment**

Congratulations! You have now removed all instances of *Payment Process* from the environment.

# Test Application

We are now ready to test the *Payment Process*.

## Review Acceptance Criteria

Let's start by reviewing the acceptance criteria that was defined earlier; we need to ensure that all of the requirements below can be met by our process.

1. **Partner** orders are processed using the **Payment Process**
2. The **Payment Process** requires a *Payment Request* which includes the *Reference*, *Amount* and *Credit Card Details*
3. The **Payment Process** processes orders automatically using a **Credit Card Service**, i.e. without human intervention

## Test Application

We can test our application and confirm that the acceptance criteria above has been met by launching a single process instance.

### *Run the Application*

Running the final version of our *PaymentApplication* will ...

- Create a new **Process Instance**
- Register a **Job Worker** for the *chargeCreditCard* task type
- Pass the *PaymentRequest* **Process Variables** into the **Job Worker**
- Invoke the 'CreditCardService'
- Return the *Confirmation* as a **Process Variable**
- Complete the **Charge Credit Card** task

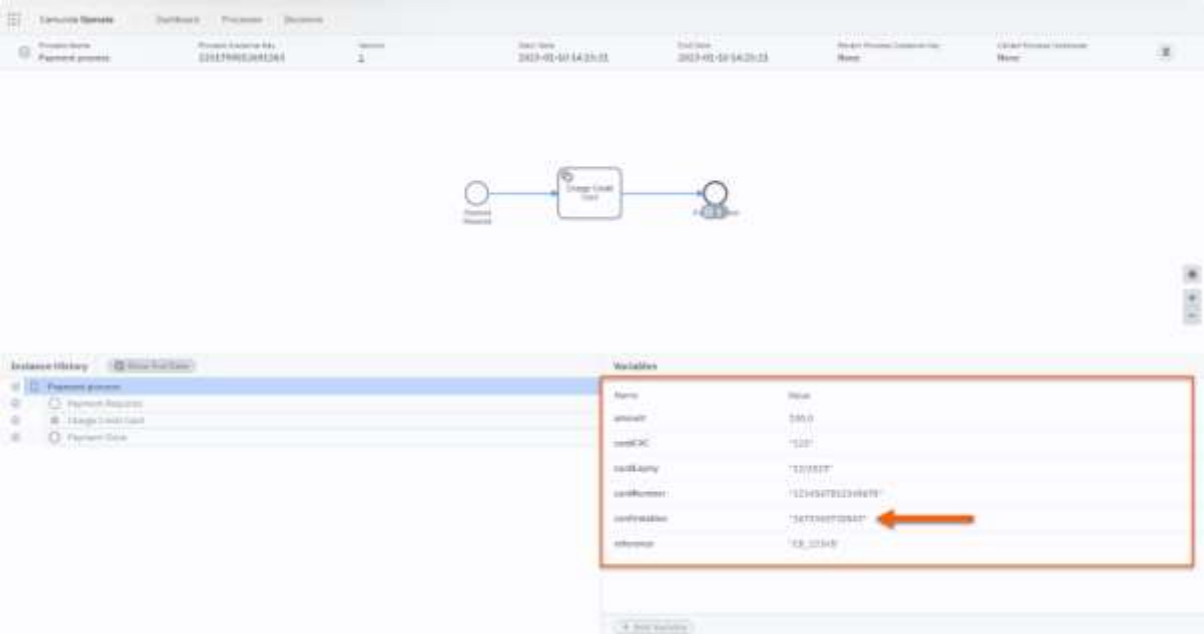
If you now run the *PaymentApplication* from your IDE or Maven, you should see output similar to the below:

```
Starting Transaction: C8_12345
Card Number: 1234567812345678
Card Expiry Date: 12/2023
Card CVC: 123
Amount: 100.0
Successful Transaction: 1673360720543
```

### *View Process Instances*

View the process instance in **Operate** by following the steps below:

1. Launch **Operate** by following the steps in the previous lesson. Alternatively, refresh the window if you still have **Operate** open.
2. Click on *Payment Process* from the **Instances by Process** panel on the left-hand side of the screen
3. **Operate** will now show a completed instance of our *Payment Process* along with the variables representing our *Payment Request*. The *Charge Credit Card* **Service Task** is now complete; you can see this from the presence of the grey marker against the **End Event**.



**Success**

Congratulations! You have now completed the testing of your process and have confirmed that it meets the acceptance criteria defined at the start of this course.

# Example Solution

## Solution

If you encountered any problems during this course or were unable to complete the process configuration, you can compare your solution with our example solution below.

### *GitHub Repository*

Our version of the application (Process Definition & Job Worker) can be found in the GitHub Repository below:

- Solution - Payment Application

### *Process Diagram*

Our version of the **Process Diagram** can be found below:

Payment Required
Charge Credit Card
Payment Done